

Web security

الدليل العملي للمختبر الأول: أسرار الشبكات،
إدارة الجلسات، واستخراج الثغرات

Web Security أمن الويب

أمن الويب هو مجموعة من الممارسات والتقنيات التي تهدف إلى حماية تطبيقات الويب من الهجمات الإلكترونية. يشمل ذلك تأمين البيانات، المصادقة، والتحكم في الوصول، وضمان سرية وسلامة وتوفير المعلومات.

Web Security is a set of practices and technologies designed to protect web applications from cyber attacks. It includes securing data, authentication, access control, and ensuring the confidentiality, integrity, and availability of information.

- The Goals of web security : CIA

How the Web Works آلية عمل الويب

Basic Web Components:

- 1.Client:** The browser (Chrome, Firefox) that sends requests to the server.
- 2.Server:** The computer hosting the website that processes requests and returns responses.
- 3.HTTP/HTTPS Protocol:** The language clients and servers use to communicate, defining how requests and responses are formatted.

مكونات الويب الأساسية:

- ١- العميل وهو المتصفح مثل الكروم او الفيرفكس يقوم بارسال طلبات الى الخادم.
- ٢- السيرفر وهو الكمبيوتر الذي يستضيف موقع الويب الذي يستقبل الطلبات ويعيد الاستجابة .
- ٣- البروتوكول وهي اللغة التي يتحدث بها العميل والخادم و يحدد كيفية تنسيق الطلبات والاستجابات مثل

HTTP/HTTPS .

آلية طلب واستجابة HTTP

العميل (Client)



المتصفح (Browser)

1 يكتب المستخدم عنوان URL

يكتب المستخدم عنوان URL



2 يُرسل طلب HTTP



يستلم الاستجابة

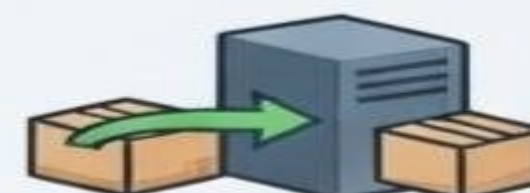


4 يعرض الصفحة (HTML, صور)

الخادم (Server)



خادم الويب (Web Server)



1 يستلم الطلب



2 يُعالج الطلب



3 يُرسل استجابة HTTP

طلب HTTP (HTTP Request)

- طريقة: GET
- المسار: /
- الرؤوس (Headers): Host, User-Agent, Cookie, Authorization...

استجابة HTTP (HTTP Response)

- رمز الحالة: 200 OK
- النوع: text/html
- الرؤوس: Date, Server...

معلومات إضافية

- Cookies
- Authorization Token
- User-Agent
- Referer

أبعاد أمن الويب: أين تقع الثغرات؟

أمن العميل (Client-side Security)

حماية المستخدم النهائي.

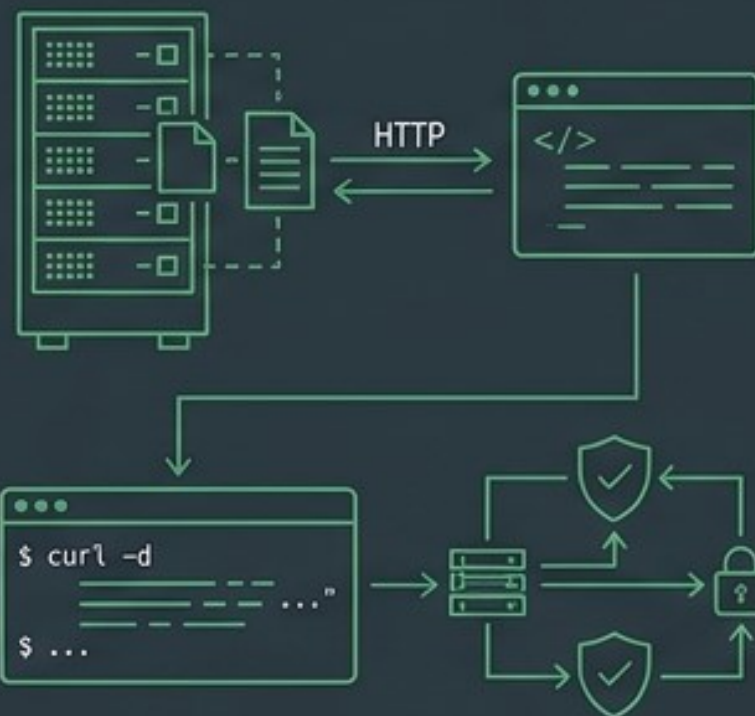
المسؤولية: منع الهجمات أثناء استخدام تطبيق الويب، مثل الهندسة الاجتماعية، تسريب البيانات الخاصة، والمتتبعات (Trackers).



أمن الخادم (Server-side Security)

لا تثق أبداً بمدخلات العميل.

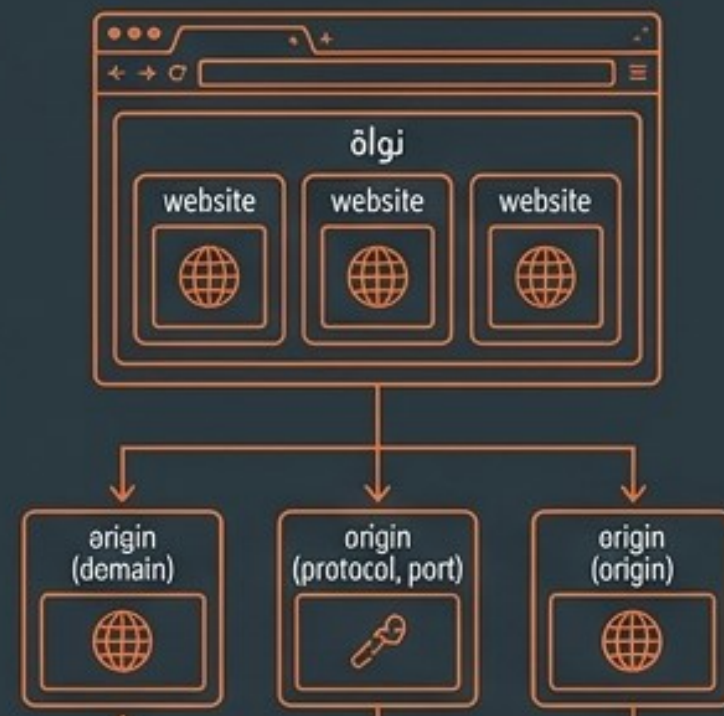
المسؤولية: المهاجمون يمكنهم تشغيل أي عميل HTTP عشوائي وإرسال أي بيانات (مثل استخدام curl -d). يجب حماية البيانات وصلاحيات الوصول.



أمن المتصفح (Browser Security)

يعمل كـ "نواة" نظام التشغيل للويب.

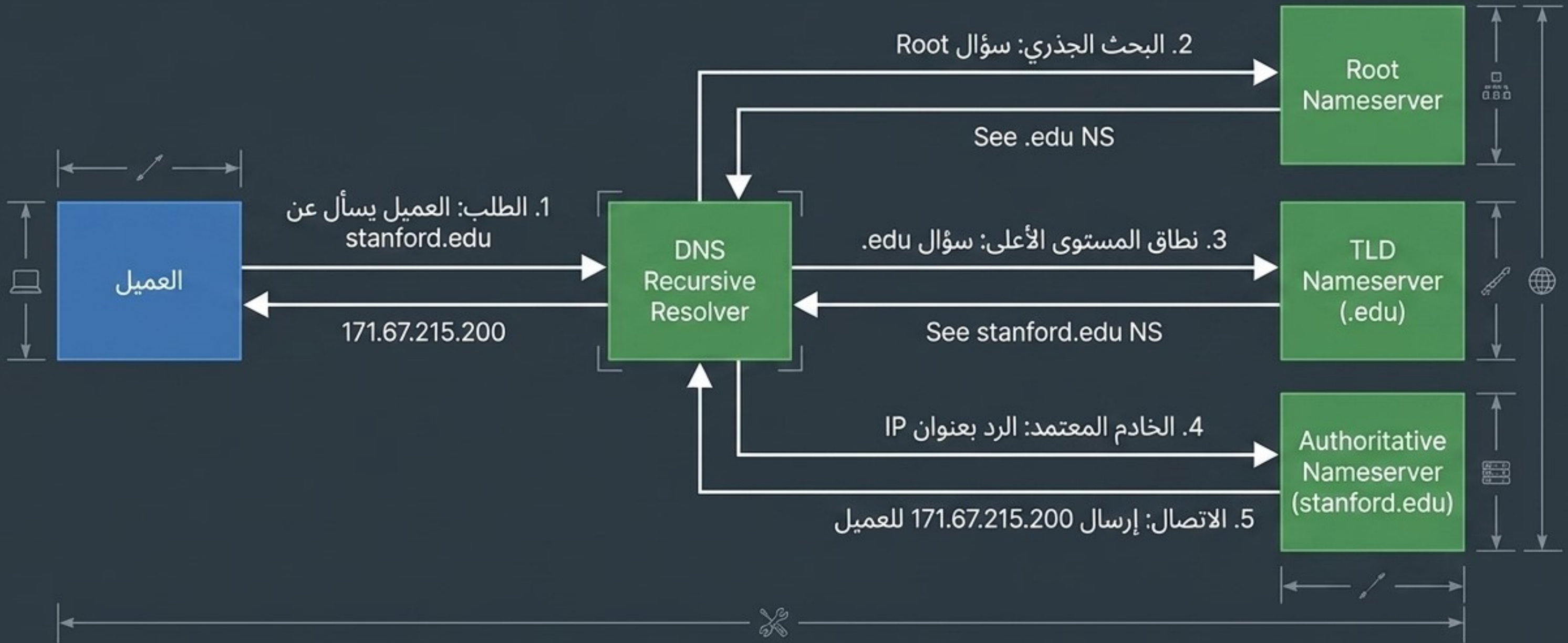
المسؤولية: عزل المواقع عن بعضها البعض (مثل سياسة نفس المصدر SOP) حتى عند تشغيلها في نفس المتصفح.



125 mm

125 mm

رحلة البحث عن العنوان: كيف يعمل نظام أسماء النطاقات (DNS)؟



بروتوكول HTTP واستخراج البيانات عبر (DevTools)

طبيعة البروتوكول

- نصي يمكن قراءته (Human-readable)
- قابل للتوسيع عبر الترويسات (Headers)
- يعمل بنظام (Stateless)؛ حيث لا توجد علاقة أي طلبين متتاليين

أدوات المطور (DevTools)

- التبويب (Network): لمراقبة الطلبات، استخراج ترويسات مثل User-Agent
- التبويب (Application): لاستخراج ملفات تعريف الارتباط (Cookies) ومعرفات الجلسة



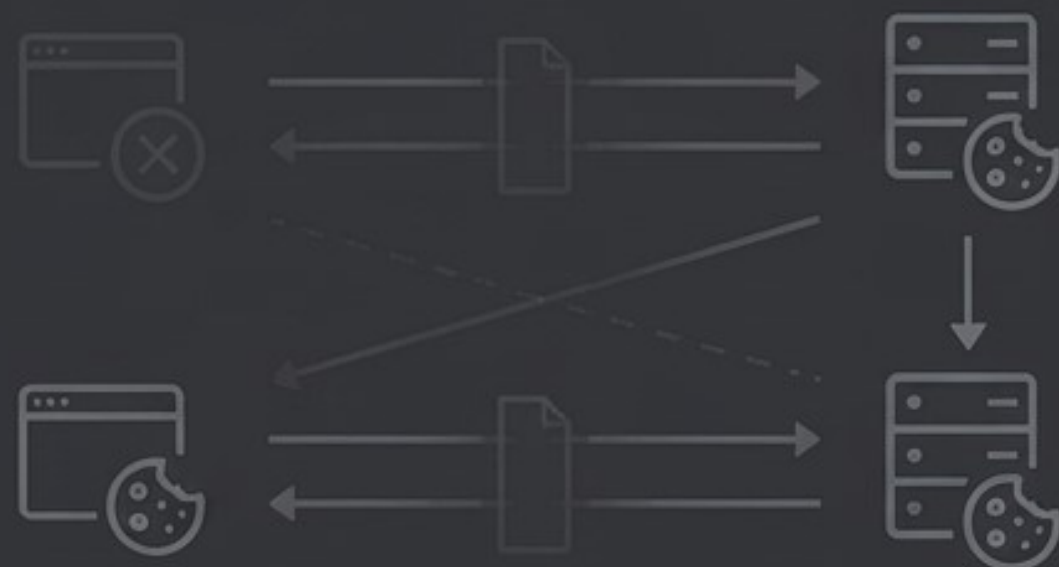
Request URL: http://example.com/ Request Method: GET Status Code: 200 OK Remote Address: 93.184.216.34:80		
▼ Response Headers Content-Type: text/html; charset=UTF-8 Date: Tue, 24 Sep 2019 01:00:27 GMT Server: ECS (oxr/8325)		
▼ Request Headers Accept-Encoding: gzip, deflate Content-Type: text/html User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/77.0.3865.90 Safari/537.36 Accept-Encoding: gzip, deflate		

معضلة الذاكرة: كيف يتذكر الخادم؟

بروتوكول عديم الحالة (Stateless)

يعامل الخادم كل طلب HTTP كأنه من مستخدم جديد تماماً.

لا توجد ذاكرة للأحداث السابقة (لا تسجيل دخول، لا سلة مشتريات).



إدارة الجلسات (Stateful via Cookies)

الخادم يرسل: **Set-Cookie: theme=dark;**

العميل يرد في كل طلب قادم:
Cookie: theme=dark;

الصلاحية المحيطة (Ambient Authority):
التحكم في الوصول بناءً على خاصية عالمية ومستمرة
ومستمرة (الكوكيز) يتم إرسالها تلقائياً مع كل طلب.

Set-Cookie: theme=dark;

Header Name

Cookie Name

Cookie Value

Cookie: theme=dark;

Header Name

Cookie Name

Cookie Value

OWASP Top 10



Broken Access Control

failures to enforce user permissions and access levels



Security Misconfiguration

incorrect or insecure configuration of systems or cloud services



Software Supply Chain Failures

compromises in the processes and components that build and update software



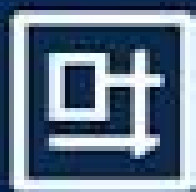
Cryptographic Failures

weak or missing cryptography leading to exposure of sensitive data



Injection

flaws that allow attackers to inject malicious code/commands



Insecure Design

security weaknesses from design flaws or lack of security controls in design



Authentication Failures

broken user authentication or session management



Software or Data Integrity Failures

integrity violations in software systems or data



Logging and Alerting Failures

inadequate logging/monitoring to detect to respond to incidents



Mishandling of Exceptional Conditions

improper error and exception handling

خريطة التهديدات: قائمة (OWASP Top 10) لعام 2025

1. نظام التحكم في الوصول المعطل
(Broken Access Control)
[مستهدف في المختبر]



2. سوء تكوين الأمان



3. إخفاقات سلسلة توريد البرمجيات



4. حالات فشل التشفير



5. الحقن (Injection)



6. التصميم غير الآمن



7. حالات فشل المصادقة
(Authentication Failures)
[مستهدف في المختبر]



8. حالات فشل سلامة البرامج أو البيانات



9. تسجيل حالات الفشل والتنبيه



10. سوء التعامل مع الظروف الاستثنائية



من الثغرة إلى الحل: إدارة الجلسات برمجياً

الفرق بين النظام الآمن والضعيف

النظام المخترق (Vulnerable)

تخزين هوية المستخدم مباشرة في الكوكي.

```
res.cookie('username', username)
```



المشكلة: يمكن للعميل تعديل الكوكي وتزوير هويته بسهولة، لأن الخادم يثق بقيمة الكوكي ثقة عمياء.

النظام الآمن (Secure Session Management)

إنشاء معرف جلسة (Session ID) عشوائي وغير متوقع.

```
const { randomBytes } = require('crypto')
```

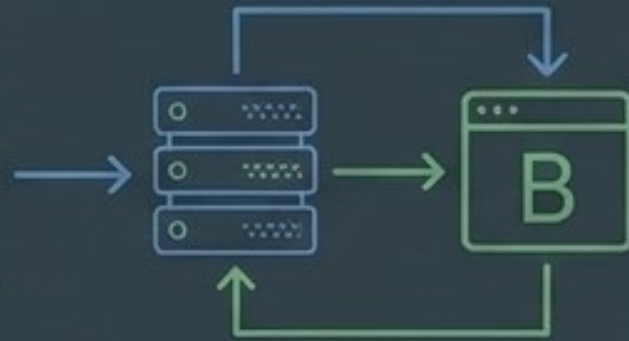


تخزين العلاقة بين المعرف العشوائي واسم المستخدم في ذاكرة الخادم (SESSIONS object).

الحل: حتى لو تلاعب العميل بالكوكي، فلن يخمن المعرف الصحيح لجلسة مستخدم آخر.

تجهيز غرفة العمليات: بيئة المختبر المحلي

لتنفيذ الهجمات ودراسة الثغرات بشكل آمن، نحتاج إلى بناء محاكي لخوادم الويب:



بيئة (Node.js)

تُستخدم لبناء خوادم HTTP سريعة وحديثة (مثل Express).

تسمح لنا بالتحكم الدقيق في الترويسات، إدارة الجلسات برمجياً، وفهم كيفية معالجة الخادم لطلبات العميل بشكل مباشر.



حزمة (XAMPP)

توفر بيئة متكاملة تشمل (Apache, MySQL).

ممتازة لمحاكاة التطبيقات التقليدية واستضافة قواعد البيانات لاختبار ثغرات الحقن وسوء الإعدادات.

تلاعب المعرفات (ID Manipulation)

كيف تحدث الثغرة؟ عندما يعتمد الخادم بشكل كامل على مُدخلات العميل (مثل الكوكيز أو المعلومات) لتحديد هوية المستخدم دون التحقق من صحتها في الخلفية.

الاستغلال: يقوم المهاجم باعتراض الطلب عبر (DevTools) وتعديل قيمة الكوكي (مثال: تغيير username=alice إلى username=admin).

النتيجة: يمنح الخادم المهاجم 'الصلاحيات المحيطة' الخاصة بالمستخدم الضحية، مما يؤدي إلى اختراق حسابات أو تجاوز صلاحيات (Broken Access Control).

Cookie: username=alice



Cookie: username=alice

Cookie: username=admin



Cookie: username=admin



جوهر العقلية الأمنية: خطر الثقة الضمنية (Implicit Trust)

كل الثغرات التي درسناها تنبع من فشل معماري واحد: افتراض الثقة.



قاعدة الهاكر والمصمم: الأمن الفعال لا يعتمد على إخفاء البيانات، بل على وضع حدود صارمة (SOP) والتحقق المستمر من كل تفاعل في الخادم.

الجدار الناري للمتصفح: سياسة نفس المصدر (SOP)

القاعدة الذهبية لأمن الويب: لا يُسمح لصفحتين من مصادر مختلفة بالتداخل مع بعضهما البعض.

ما هو المصدر (Origin)؟

تتكون هوية المصدر من مطابقة تامة لثلاثة عناصر:



لا يُسمح لسكربت يعمل في سياق مصدر معين بالوصول إلى بيانات مصدر آخر،
وإلا لاستطاع أي موقع خبيث قراءة بياناتك من تبويب آخر!

أهداف المختبر الأول: بدء التنفيذ

أنت الآن جاهز للانتقال من الجانب النظري إلى التطبيق العملي. مهامك في المختبر:

- [] إعداد بيئة (Node.js) و (XAMPP) محلياً.
- [] استخدام (DevTools) لاعتراض وتعديل طلبات HTTP والكوكيز.
- [] تنفيذ هجوم تلاعب المعرفات (ID Manipulation) لاختراق حساب.
- [] كتابة كود برمجي آمن (Secure Session Management) لترقيع الثغرة.

تذكر: لا تفترض الأمان.. قم ببنائه.